



Trajecto

Équipe 10

Étudiants :

Benjamin Blanc
Manon Duquesne
Hugo Lhommeau
Jérémy L'Hostis
Alexandre Yu

Encadrant :

Kévin Leroy

Table des matières

Introduction	1
1 Planning	2
1.1 Phase d'analyse	2
1.2 Phase de développement	3
2 Fonctionnalités principales	4
2.1 Gestion des comptes utilisateur	4
2.2 Gestion des lignes d'exploitation	5
2.3 Gestion des trajets proposés	5
2.4 Recherche d'un trajet	6
2.5 Positionnement commercial	7
3 Modélisation	7
3.1 Architecture générale	7
3.2 Diagrammes des cas d'utilisation	9
3.3 Diagrammes d'activité	10
3.4 Diagramme de classes	14
3.4.a Couche objet {unlisted}	14
3.5 Modèle de base de données	16

Introduction

Et si chacune et chacun pouvait imaginer sa propre manière de voyager en train ? C'est autour de cette idée que s'articule **Trajecto**, une plateforme destinée à proposer, gérer et rechercher des trajets ferroviaires en s'appuyant sur les données du réseau SNCF.

L'objectif de notre application est de fournir une plateforme permettant à une entreprise ferroviaire de gérer son offre de transport et à ses clients de rechercher facilement les trajets disponibles. L'interface graphique étant prise en charge par les webmasters de l'entreprise, notre travail se concentre sur la conception et le développement d'une API robuste, sécurisée et exploitable par différents clients.

L'application s'articule principalement autour de quatre fonctionnalités : de la gestion des comptes utilisateurs, à trois niveaux d'habilitation différents, à la recherche de trajets précis, en passant par l'organisation des lignes d'exploitation.

En complément de ces fonctionnalités essentielles, nous étudierons un positionnement commercial propre à notre entreprise afin de proposer une expérience différente de celle du réseau ferroviaire traditionnel. Cette fonctionnalité pourra notamment s'appuyer sur une meilleure transparence des informations proposées au client, par exemple en détaillant la composition du prix d'un trajet.

Ce rapport d'analyse intermédiaire présente ainsi les choix effectués pour répondre à ces besoins. Nous y décrirons le fonctionnement général de l'application, les différents acteurs et leurs interactions avec l'API, la structure des données ainsi que les principaux choix d'implémentation envisagés. L'objectif est de disposer, à l'issue de cette phase, d'une vision suffisamment précise du système pour pouvoir passer au développement de l'application dans de bonnes conditions.

1 Planning

Voici le planning prévisionnel de la réalisation de ce projet, sous forme classique de diagramme de Gantt, incluant les tâches déjà effectuées et les échéances qui guideront notre organisation au cours des tâches futures :

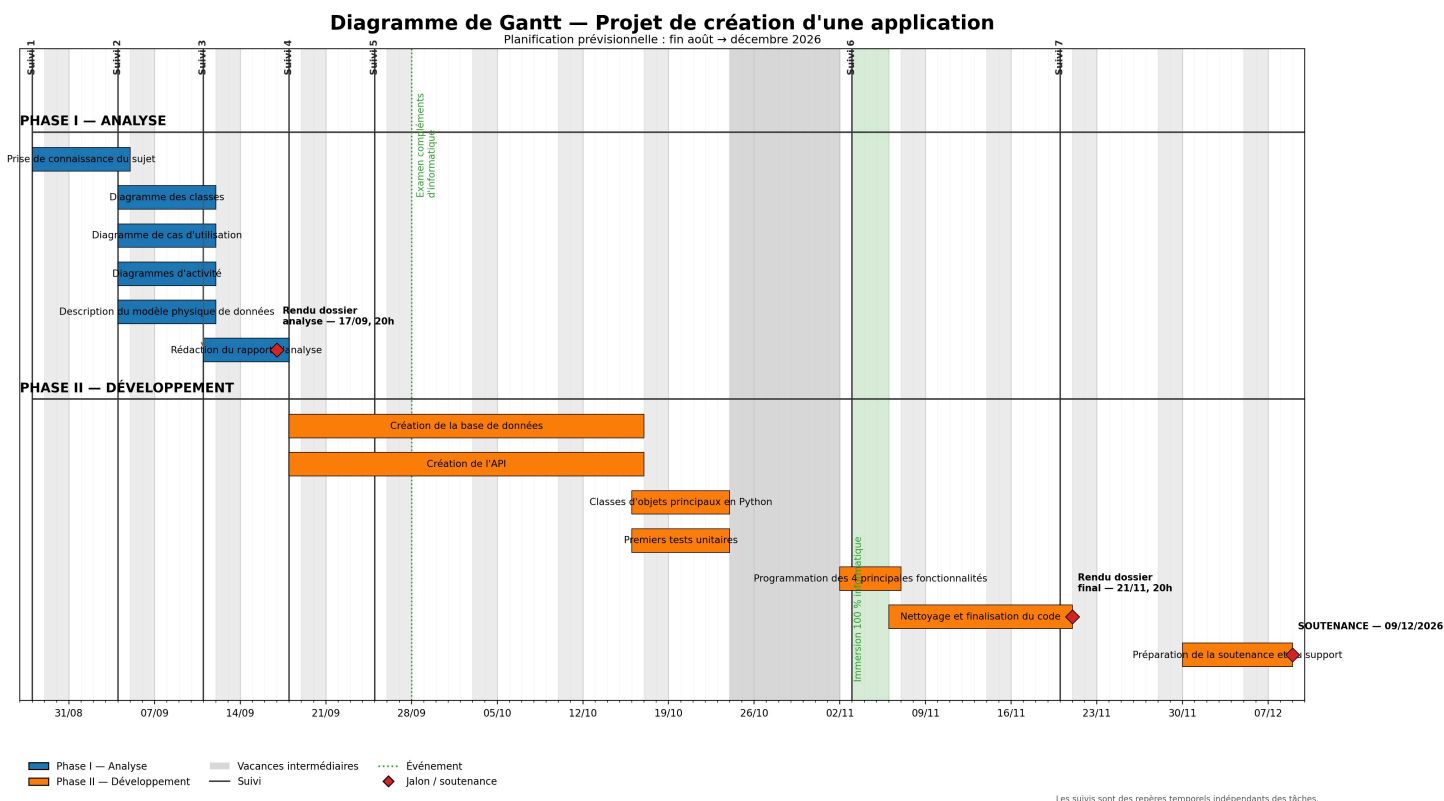


Figure 1: Planning

1.1 Phase d'analyse

La première phase du projet, qui a duré trois semaines environ, a consisté à prendre connaissance des consignes exactes et de ce qu'elles impliquaient en termes de choix technologiques. À ce titre, les premiers travaux pratiques, ayant eu lieu en parallèle de façon hebdomadaire, ont été une aide précieuse à la compréhension du sujet, même s'il nous reste à ce jour des réflexions à achever, des points à préciser et des décisions à prendre (voir notamment Section 3.4).

Nous en avons profité pour faire connaissance et pour répartir les tâches indiquées sur la première semaine selon nos appétences et connaissances respectives. Le diagramme des classes, plus complexe, a cependant occupé plusieurs d'entre nous, de même que la rédaction du présent rapport.

Cette phase avait donc pour but principal de cibler nos objectifs de travail et d'assimiler les connaissances informatiques idoines : notions de couches, services, l'ensemble concepts liés aux bases de données, et le fonctionnement de l'API SNCF, nécessaire pour récupérer les données nous permettant, à terme, de gérer les lignes d'exploitation de Trajecto.

Bien entendu, cette phase a aussi servi à organiser le groupe et à effectuer une projection dans le temps des tâches futures; c'est-à-dire celles de la phase de développement.

1.2 Phase de développement

Cette seconde phase, qui durera deux mois et demi, s'articulera en trois temps.

- *Initialisation* : Entre le rendu du présent rapport et le congé de la Toussaint, cinq semaines s'écouleront. Cette période contient également les deux derniers travaux pratiques, qui se concentreront sur l'expérience utilisateur. Elle contient enfin l'examen de "Compléments d'informatique", l'UE associée au projet. Par conséquent, nous mobiliserons toute la durée pour parfaire nos connaissances nécessaires à la programmation des fonctionnalités et faire deux choses importantes : créer la base de données de notre application (et son s'assurer de la cohérence de sa structure), et programmer les composants les plus fondamentaux de notre API, comme la fonction de création de compte utilisateur par exemple. La dernière semaine sera consacrée à toutes les classes de la couche objet et aux tests unitaires dédiés auxdits objets : elle nous permettra de nous rendre compte des éventuelles corrections à apporter à notre modélisation.
- *Programmation* : À la rentrée de novembre, trois jours seront consacrés à une immersion totale dans le projet. Cette période dense, dont l'organisation précise reste à définir, servira à programmer la majorité des quatre fonctionnalités (selon notre efficacité, peut-être la cinquième) que nous présentons dans la Section 2. À son issue, notre application devra être, sinon totalement, du moins partiellement prête à l'usage.
- *Finalisation* : Pendant les deux semaines qui suivront, il s'agira en parallèle de corriger les bugs, finaliser les quelques algorithmes manquants, et rédiger le rapport final. Enfin, trois semaines resteront et permettront de préparer la présentation orale de notre application en complément du rapport. Ce sera éventuellement l'occasion de créer une frontend attractive, quelques fonctionnalités facultatives, et de générer des exemples d'utilisation convaincants.

2 Fonctionnalités principales

2.1 Gestion des comptes utilisateur

2.1.a Description

Permettre aux utilisateurs de créer et gérer leur compte. Chaque compte possède un nom d'utilisateur, un mot de passe et un niveau d'habilitation parmi CLIENT, COLLABORATEUR et ADMIN. Les comptes ADMIN peuvent modifier le niveau d'habilitation des autres utilisateurs, à l'exception des autres comptes ADMIN.

2.1.b Fonctionnement

Lors de la création d'un compte, l'utilisateur renseigne un nom d'utilisateur et un mot de passe. Le compte est associé à un niveau d'habilitation CLIENT.

Un compte ADMIN peut consulter les comptes utilisateurs et augmenter ou diminuer leur niveau d'habilitation. Un ADMIN ne peut cependant pas modifier le niveau d'un autre ADMIN.

Un compte est associé à un identifiant unique, qui est utilisé pour identifier le compte dans la base de données.

Un ADMIN ne peut pas modifier le niveau d'habilitation d'un autre ADMIN.

Un utilisateur non ADMIN ne peut pas modifier les habilitations des autres comptes.

(Le mot de passe doit être stocké de manière sécurisée et ne doit jamais être enregistré en clair.)

2.1.c Prérequis

- Nom d'utilisateur
- Mot de passe
- Niveau d'habilitation lors de la création ou de la modification
- Identifiant du compte à modifier pour un ADMIN

2.1.d Tâches effectuées

- Création d'un nouveau compte utilisateur
- Connexion à un compte existant
- Modification du niveau d'habilitation d'un utilisateur
- Affichage d'un message de confirmation ou d'erreur en cas d'échec

2.1.e Exemple

Un utilisateur crée un compte avec le nom alice et un mot de passe. Son compte est créé avec le rôle CLIENT. Un ADMIN peut ensuite faire évoluer son rôle en COLLABORATEUR. L'ADMIN ne peut toutefois pas modifier le rôle d'un autre compte ayant déjà le niveau ADMIN.

2.2 Gestion des lignes d'exploitation

2.2.a Description

Permettre à un compte COLLABORATEUR de créer, modifier et supprimer des lignes d'exploitation reliant une gare de départ à une gare de terminus. Les gares utilisées sont récupérées à partir de l'API SNCF afin de garantir leur existence.

2.2.b Fonctionnement

Lors de la création ou de la modification d'une ligne, le collaborateur recherche les gares grâce à une fonctionnalité d'autocomplétion. Les résultats proposés proviennent de l'API SNCF.

Le collaborateur sélectionne une gare de départ et une gare de terminus parmi les gares existantes, puis enregistre la ligne d'exploitation.

Les deux gares doivent exister dans les données fournies par l'API SNCF.

Une ligne ne peut pas être créée si la gare de départ et la gare de terminus sont identiques.

Un compte CLIENT ne peut pas gérer les lignes d'exploitation, sauf évolution ultérieure des droits.

La suppression d'une ligne peut être refusée si des trajets sont déjà planifiés sur cette ligne, afin de préserver l'intégrité des données.

2.2.c Prérequis

- Gare de départ
- Gare de terminus
- Compte utilisateur disposant du rôle COLLABORATEUR

2.2.d Tâches effectuées

- Création d'une nouvelle ligne d'exploitation
- Modification d'une ligne existante
- Suppression d'une ligne
- Affichage d'un message de confirmation ou d'erreur en cas d'échec

2.2.e Exemple

Un collaborateur saisit "Rennes" dans le champ de départ. L'application propose notamment "Rennes" grâce à l'API SNCF. Il sélectionne Rennes comme gare de départ et Paris comme gare de terminus, puis crée la ligne Rennes → Paris.

2.3 Gestion des trajets proposés

2.3.a Description

Permettre à un compte COLLABORATEUR de planifier des trajets sur une ligne d'exploitation existante. Un trajet est défini notamment par une date et une heure de départ, un nombre de places disponibles et un tarif. Le temps de trajet est automatiquement déterminé à partir de l'API SNCF.

2.3.b Fonctionnement

Le collaborateur sélectionne une ligne d'exploitation existante, puis renseigne la date et l'heure de départ, le nombre de places et le tarif.

L'application interroge l'API SNCF afin de vérifier qu'un trajet en train est possible entre la gare de départ et le terminus. Elle récupère également les informations nécessaires au calcul automatique du temps de trajet.

Le collaborateur peut ensuite modifier ou supprimer le trajet planifié.

La ligne d'exploitation doit exister avant de pouvoir planifier un trajet.

Le trajet doit être réalisable en train selon les informations fournies par l'API SNCF.

Le temps de trajet ne doit pas être saisi manuellement par le collaborateur.

La date et l'heure de départ doivent être valides.

Le nombre de places doit être supérieur à zéro.

Le tarif doit être positif ou nul selon les règles commerciales retenues.

Un trajet ne peut pas être planifié si l'API SNCF ne permet pas de vérifier sa faisabilité.

2.3.c Prérequis

- Ligne d'exploitation
- Date de départ
- Heure de départ
- Nombre de places
- Tarif
- Compte utilisateur disposant du rôle COLLABORATEUR ou ADMIN

2.3.d Tâches effectuées

- Création d'un trajet planifié
- Temps de trajet calculé automatiquement
- Modification ou suppression d'un trajet
- Confirmation ou message d'erreur

2.3.e Exemple

Un collaborateur sélectionne la ligne Rennes → Paris, choisit le 15 octobre à 08h00, indique 50 places disponibles et un tarif de 45 €. L'API SNCF confirme qu'un trajet est possible et permet à l'application de calculer automatiquement sa durée.

2.4 Recherche d'un trajet

2.4.a Description

Permettre à un compte CLIENT de rechercher les trajets proposés entre une gare de départ et une gare d'arrivée. Une autocomplétion facilite la sélection des gares.

2.4.b Fonctionnement

Le client saisit une gare de départ et une gare d'arrivée. L'application propose automatiquement des gares correspondant à la saisie grâce aux données issues de l'API SNCF.

Après validation de la recherche, l'application affiche les trajets proposés correspondant aux critères renseignés.

Pour chaque trajet, le client peut consulter la gare de départ, la gare d'arrivée, la date et l'heure de départ ainsi que le temps de trajet.

Les gares doivent correspondre à des gares existantes.

La gare de départ et la gare d'arrivée doivent être différentes.

Si aucun trajet ne correspond à la recherche, un message explicite est affiché.

Une recherche peut retourner plusieurs trajets pour une même liaison.

2.4.c Prérequis

- Gare de départ
- Gare d'arrivée
- Compte utilisateur actif, c'est-à-dire disposant du rôle CLIENT au minimum

2.4.d Tâches effectuées

Renvoie une liste de trajets correspondant aux gares recherchées, comprenant au minimum : - Gare de départ - Gare d'arrivée - Date de départ - Heure de départ - Temps de trajet

2.4.e Exemple

Un client sélectionne Rennes comme gare de départ et Paris comme gare d'arrivée. L'application affiche les trajets disponibles entre ces deux gares avec leur date, leur heure de départ et leur durée.

2.5 Positionnement commercial

2.5.a Description

Proposer une fonctionnalité permettant de différencier le service du réseau commercial de la SNCF en offrant au client une tarification transparente. Cette fonctionnalité ne sera implémentée que si le temps le permet: nous n'allons pas l'explorer dans ce rapport pour l'heure.

3 Modélisation

3.1 Architecture générale

3.1.a Base de données :

Pour stocker les données, nous allons utiliser une base de données postgresQL, sur l'environnement du datalab de l'ENSAI. La base de données contiendra les informations utilisateurs, ainsi que les gares, les lignes de train et les trajets prévus. Ces données seront saisies petit à petit. Ainsi, les collaborateurs

écriront dans la base de données lorsqu'ils créeront un nouveau trajet, ou lorsqu'une nouvelle ligne sera créée.

3.1.b API :

Nous allons aussi récupérer des données via l'API de la SNCF, Navitia. Navitia est une API qui fonctionne par requêtes HTTP GET. Une requête HTTP GET est une demande envoyée à un serveur pour récupérer des informations à partir d'une URL. En envoyant cette requête, l'API demande une clé d'authentification, qui est disponible gratuitement avec une limite de 1500 appels par jour, et dont nous avons fait la demande au préalable. L'API renvoie ensuite une réponse HTTP contenant les données demandées au format JSON. Dans le cadre de notre projet, nous utiliserons l'API pour deux principales raisons :

- Récupérer les gares existantes
- Récupérer les temps de trajet existants sur une ligne.

3.1.c DAO :

Les DAO, pour Data Access Object, vont nous permettre de faire le lien entre les classes métier et la base de données. Leur rôle est de gérer l'accès aux données sans que les classes métier aient besoin de connaître le fonctionnement de la base de données. Ils permettent notamment, via des requêtes SQL, d'enregistrer les objets créés par les collaborateurs (gares, trajets, utilisateurs, etc.) dans la base de données afin de les sauvegarder. Ils permettent également de récupérer les données de la base de données et de les transformer en objets Python utilisables par le reste de l'application.

3.2 Diagrammes des cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous permet de présenter simplement l'ensemble des fonctionnalités de l'application et les droits d'accès à ces dernières selon le statut de l'utilisateur.

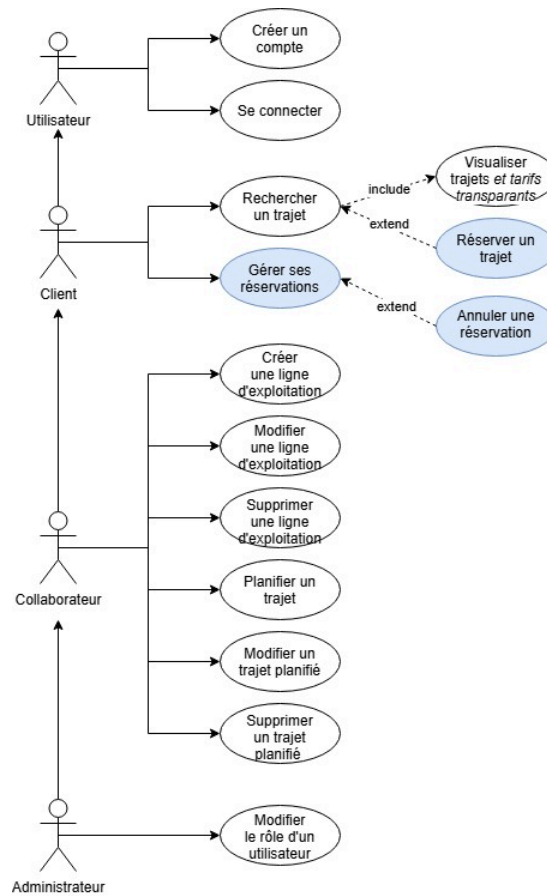


Figure 2: Cas d'utilisation

L'application s'articule autour de quatre niveaux d'habilitation, dont l'ordre croissant est le suivant : l'Utilisateur (visiteur non connecté), le Client, le Collaborateur et l'Administrateur. Chaque statut bénéficie des fonctionnalités des niveaux inférieurs tout en accédant à ses propres options. Cette organisation doit garantir une séparation claire des responsabilités : le Client effectue des recherches, réserve et gère ses trajets, pendant que le Collaborateur assure la gestion opérationnelle du réseau (création, modification ou suppression de lignes et planification, modification ou suppression des trajets). Initialement, cela nous avait amené à modéliser les quatre rôles par un héritage : chaque niveau d'habilitation supérieur était une classe fille du niveau inférieur. Nous verrons en Section 3.4.a pourquoi nous avons finalement remis ce choix en cause. De son côté, l'Administrateur se concentre sur l'attribution et la modification des rôles des utilisateurs. Enfin, les bulles colorées mettent en évidence les fonctionnalités optionnelles de l'application, telles que la réservation ou l'annulation d'un trajet pour le client, que nous ne programmerons qu'en fin de développement si nous pouvons le faire dans de bonnes conditions, notamment temporelles.

3.3 Diagrammes d'activité

3.3.a Utilisateur de base

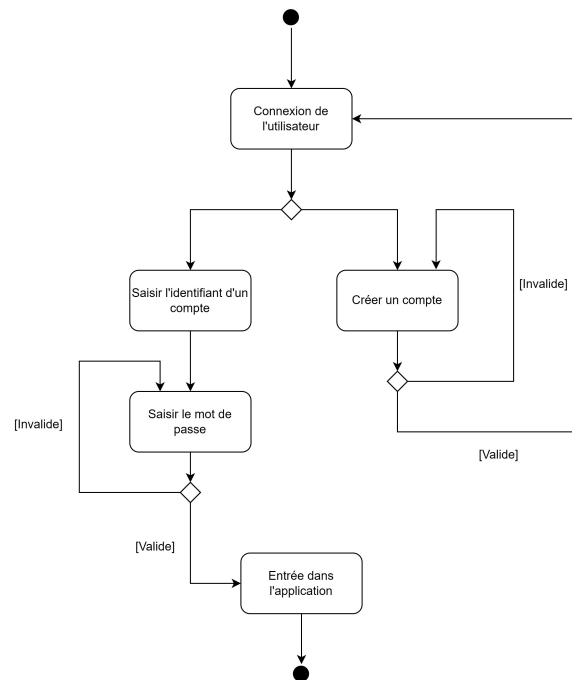


Figure 3: Activité de l'utilisateur

L'utilisateur de base accède à l'application via un point d'entrée lui offrant deux parcours distincts : la connexion à un compte existant ou la création d'un nouveau compte. S'il choisit de créer un compte, il est invité à renseigner ses informations. En cas de saisie invalide, le système lui demande de recommencer, tandis qu'une saisie valide le redirige vers l'étape de connexion. Pour se connecter, l'utilisateur saisit d'abord son identifiant puis son mot de passe. Si le mot de passe est incorrect, une nouvelle saisie lui est demandée. Une fois l'authentification validée, l'accès à l'application lui est définitivement accordé, ce qui met fin au processus.

3.3.b Client

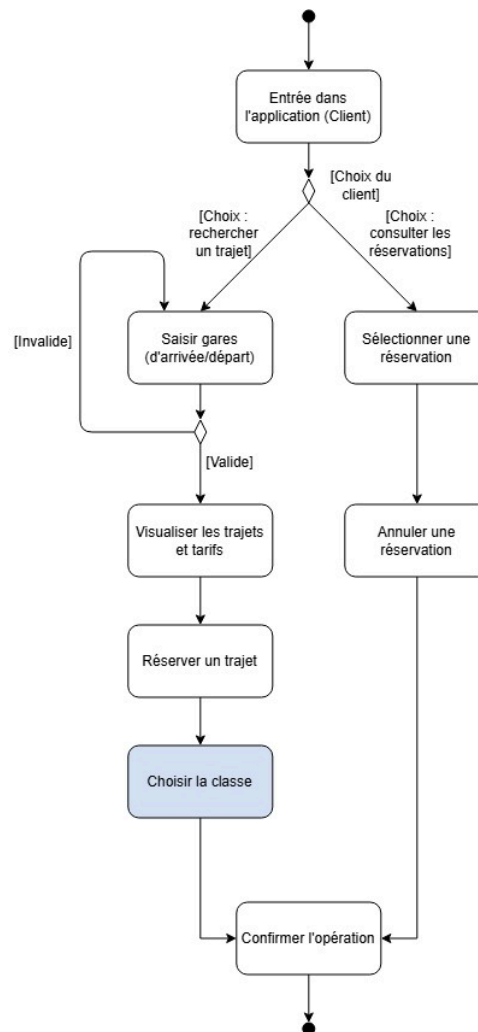


Figure 4: Activité du client

Une fois connecté en tant que Client, deux actions principales sont proposées dès l'entrée dans l'application : rechercher un trajet ou consulter les réservations existantes. Si le client choisit de rechercher un trajet, il saisit les gares de départ et d'arrivée. En cas d'erreur de saisie, le système lui demande de corriger les informations. Dès que la saisie est valide, il peut visualiser les trajets et tarifs disponibles, procéder à la réservation puis choisir sa classe (fonctionnalité optionnelle mise en valeur par la couleur). S'il opte pour la consultation des réservations, il peut sélectionner une réservation afin de l'annuler. Quel que soit le parcours emprunté, l'action se conclut par une confirmation de l'opération avant de mettre fin au processus.

3.3.c Collaborateur

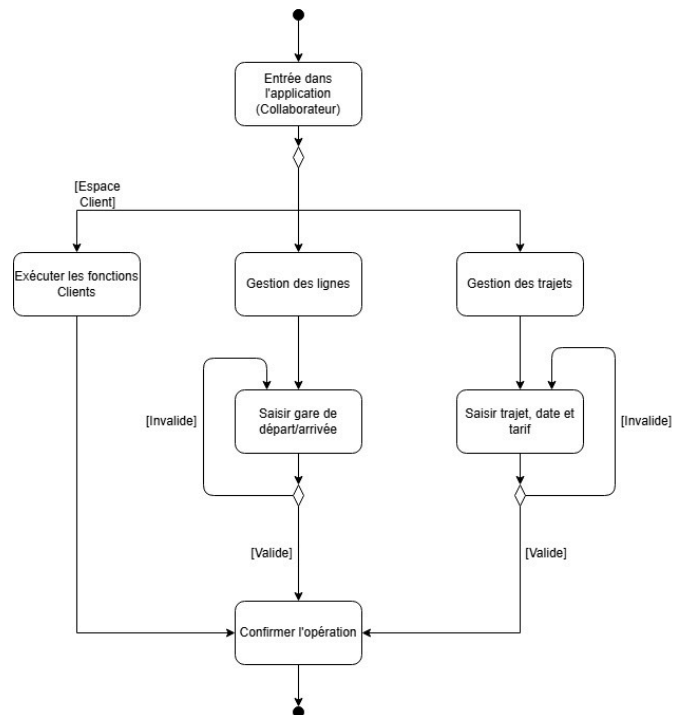


Figure 5: Activité du collaborateur

À son entrée dans l'application, le Collaborateur dispose de trois choix d'actions : accéder à l'espace client pour en exécuter les fonctionnalités, assurer la gestion des lignes ou gérer les trajets. Dans le cadre de la gestion des lignes, il doit saisir la gare de départ et d'arrivée. Si les données renseignées sont invalides, une nouvelle saisie est demandée jusqu'à validation. De manière similaire, pour la gestion des trajets, le collaborateur renseigne le trajet, la date ainsi que le tarif, avec une boucle de contrôle qui invite à recommencer en cas d'erreur. Quel que soit le parcours sélectionné, l'activité se conclut par une confirmation de l'opération avant d'achever le processus.

3.3.d Administrateur

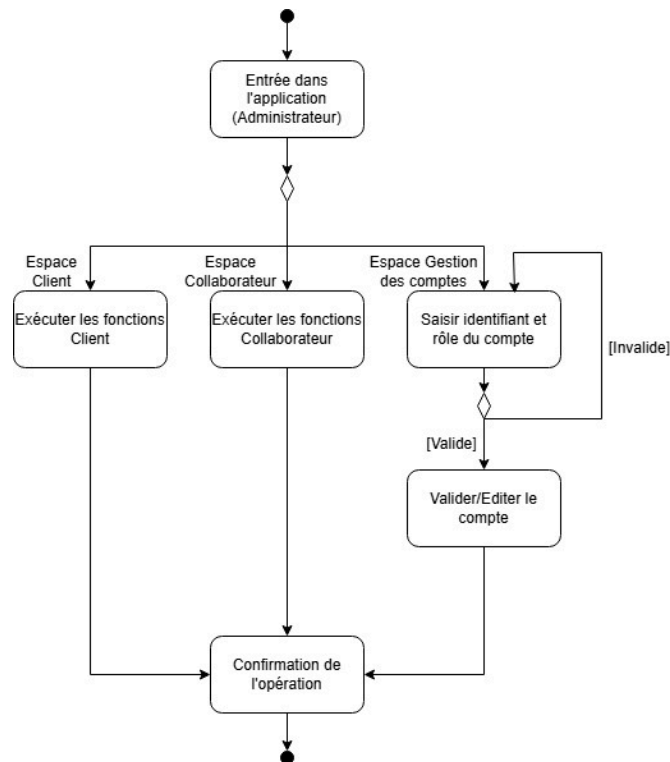


Figure 6: Activité de l'administrateur

L'administrateur a accès à trois espaces distincts : l'espace Client, l'espace Collaborateur ou l'espace Gestion des comptes. S'il choisit l'un des deux premiers espaces, il peut exécuter directement les fonctionnalités réservées respectivement aux clients ou aux collaborateurs. S'il opte pour la gestion des comptes, il saisit l'identifiant et le rôle à attribuer au compte concerné. De la même façon que précédemment, en cas de saisie invalide, le système l'invite à recommencer. Une fois les informations validées, il peut valider ou éditer le compte. Quel que soit le parcours choisi, l'activité s'achève par une confirmation de l'opération avant de terminer le processus.

3.4 Diagramme de classes

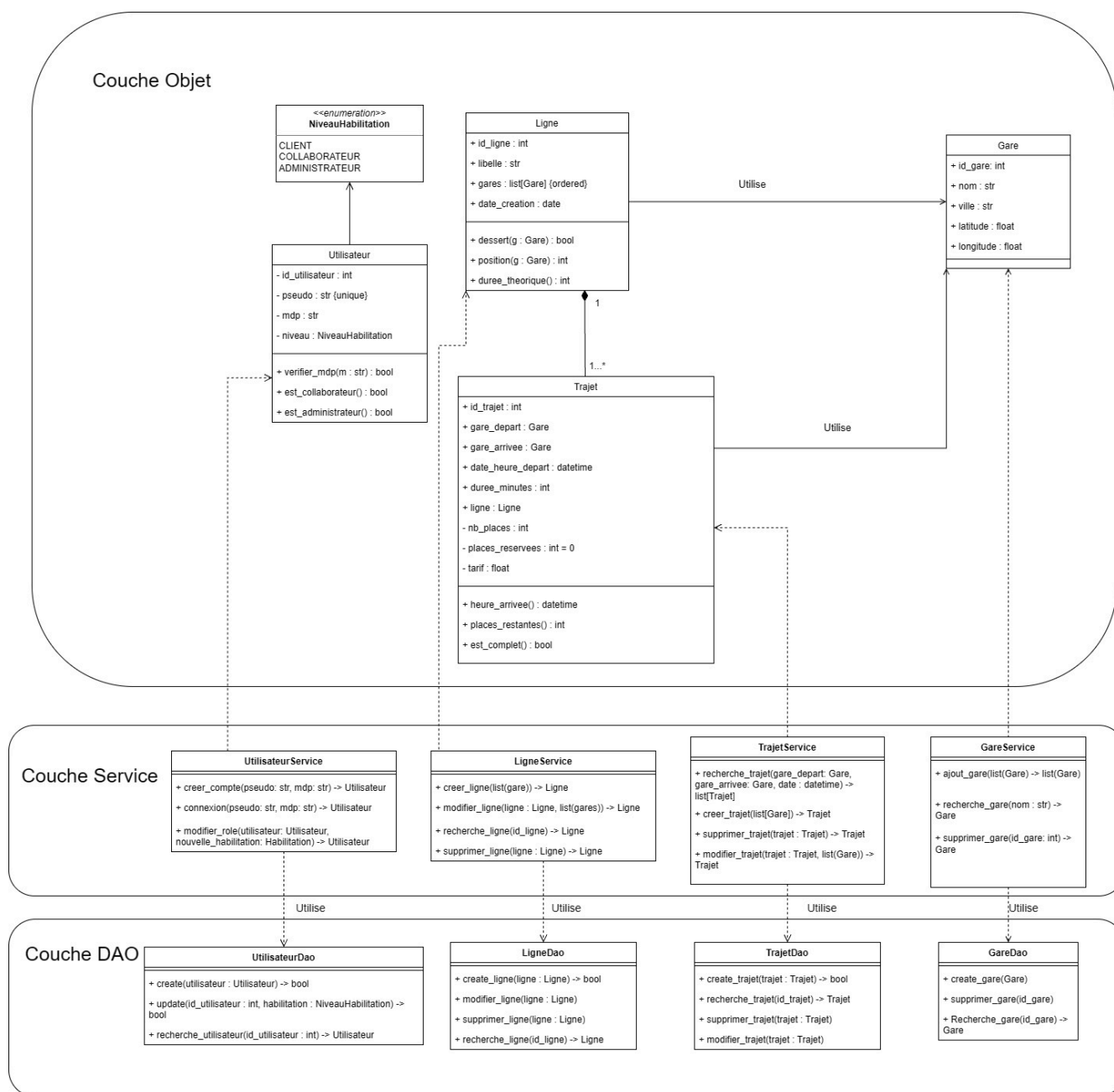


Figure 7: Classes

3.4.a Couche objet {unlisted}

Nous présentons dans cette partie le diagramme de classes de notre application, qui regroupe les objets utilisés, leurs attributs, et les méthodes qu'ils permettent de mettre en œuvre. La couche objet est composée de quatre classes principales : Utilisateur, Gare, Ligne et Trajet. La classe Utilisateur permet de représenter les différents utilisateurs de l'application grâce à son identifiant, son pseudo, son mot de passe et son niveau d'habilitation. Ce dernier peut prendre trois valeurs : CLIENT, COLLABORATEUR ou ADMINISTRATEUR. Cette énumération est présente car il est précisé dans le sujet qu'un administrateur peut modifier le niveau d'habilitation d'un utilisateur (client, collaborateur, admin). Ce choix permet notamment de modifier directement le niveau d'habilitation d'un utilisateur existant, comme le prévoit le sujet. Si chaque niveau avait été représenté par une classe différente, il aurait fallu changer la classe de l'objet lorsque son niveau d'habilitation était modifié, ce qui n'est pas possible directement en

programmation orientée objet. La classe Gare représente une gare ferroviaire et contient un identifiant, un nom, une ville ainsi que ses coordonnées géographiques. Elle permet notamment de stocker les informations nécessaires à l'identification et à la localisation d'une gare. La classe Ligne représente une ligne ferroviaire et contient un identifiant, un libellé, une liste ordonnée de gares ainsi qu'une date de création. Elle possède les méthodes `dessert()` pour vérifier si une gare appartient à la ligne et `position()` pour obtenir sa position. Une ligne est associée à une ou plusieurs gares et possède un ou plusieurs trajets. La classe Trajet représente un trajet ferroviaire et contient notamment une gare de départ, une gare d'arrivée, une date et heure de départ, une durée, un nombre de places et un tarif. Elle possède les méthodes `heure_arrivee()` pour déterminer l'heure d'arrivée, `places_restantes()` pour connaître le nombre de places disponibles et `est_complet()` pour vérifier si le trajet est complet. Ces deux dernières méthodes font partie des fonctionnalités optionnelles de l'application, donc elles ne seront peut être pas implantées dans la version finale. Les relations entre les classes permettent de représenter les dépendances entre les différents objets : une Ligne utilise des Gare, tandis qu'un Trajet utilise une gare de départ et une gare d'arrivée et est associé à une Ligne. L'ensemble de ces classes constitue ainsi la partie métier de l'application et sert de base aux couches service et DAO, qui les utilisent pour réaliser les différentes fonctionnalités de l'application.

La plupart des attributs et toutes les méthodes de ces classes sont, du moins pour le moment, modélisées comme étant publics. En effet, les interactions extérieures seront très fréquentes. Les deux exceptions correspondent à la gestion interne des propriétés des objets sur lesquelles personne d'autre ne peut intervenir : les attributs d'identification et de connexion de l'utilisateur qui sont privés, et, si nous implémentons effectivement les fonctionnalités correspondantes, les attributs donnant les informations de réservation de chaque train, également privés.

3.4.b Couche service

La couche service se situe entre l'interface de l'application et les couches objet et DAO. Elle contient ici `UtilisateurService`, `LigneService`, `TrajetService` et `GareService`. Son rôle est de gérer les fonctionnalités de l'application en utilisant les objets métiers et les DAO. Par exemple, `LigneService` permet de créer, modifier ou supprimer une ligne, tandis que `TrajetService` permet de rechercher un trajet, d'en créer un ou de le modifier. Les méthodes de cette couche sont publiques car elles constituent les fonctionnalités utilisables par le reste de l'application. On observe deux dépendances principales dans le diagramme : les services utilisent les objets métiers comme paramètres ou valeurs de retour (`Ligne`, `Gare`, `Trajet`, `Utilisateur`) et ils utilisent également les DAO pour accéder aux données de la base de données.

3.4.c Couche DAO

La couche DAO, pour Data Access Object, constitue une interface entre la couche service et la base de données. Elle regroupe les différentes opérations nécessaires à la gestion des données de l'application. Dans notre projet, elle est composée de trois classes : `UtilisateurDao`, `LigneDao` et `TrajetDao`, chacune étant associée aux objets métier correspondants. Les méthodes de ces classes permettent principalement de convertir les données récupérées depuis la base de données, initialement au format JSON, en objets Python utilisables par le reste de l'application, et inversement, de transformer les attributs des objets en paramètres utilisables dans les requêtes SQL.

Pour la même raison que dans la couche objet, les attributs de ces classes sont tous publics.

3.5 Modèle de base de données

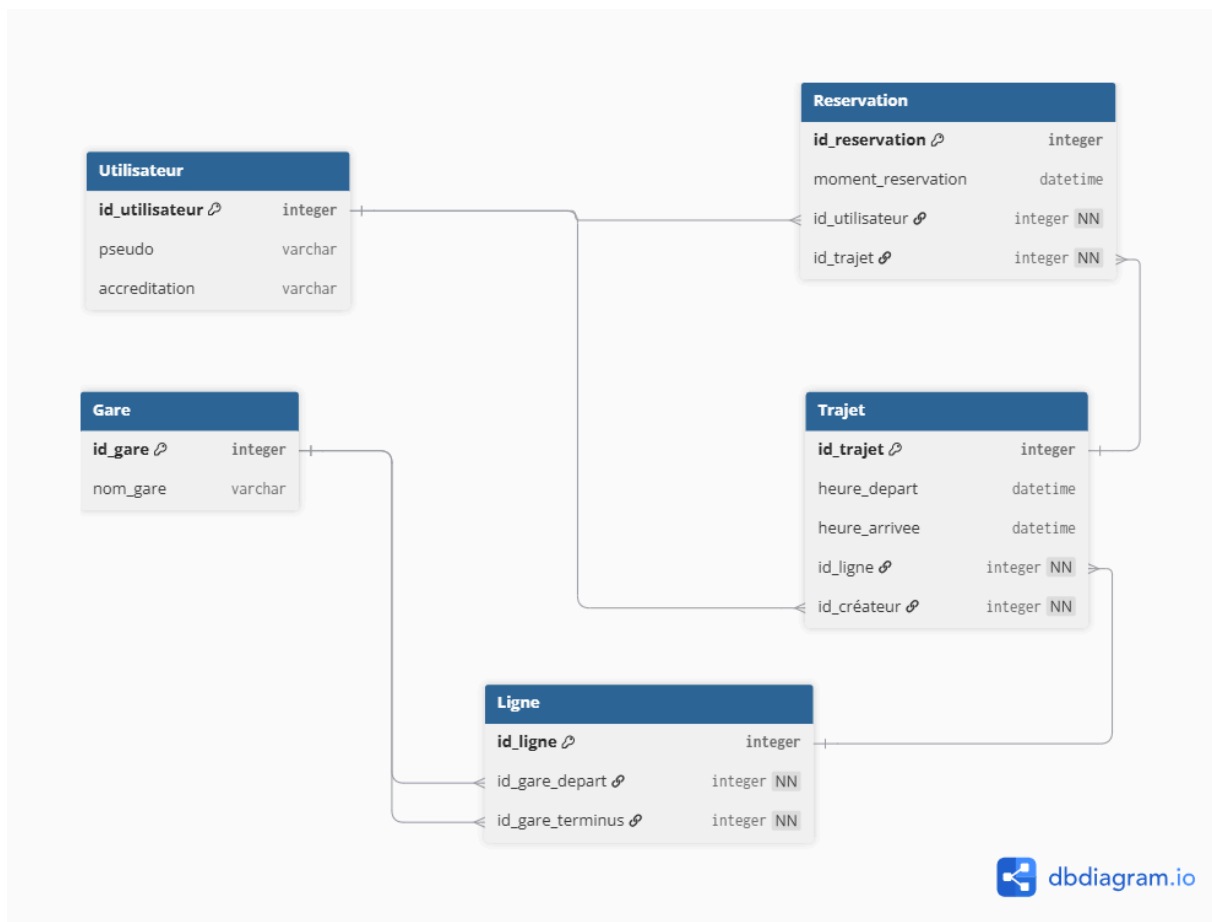


Figure 8: Base de données

Le diagramme de base de données (MLD) suit le modèle du diagramme de classe. Chaque classe objet principal correspond à une table de la base de données, avec une table Réservation, qui ne sera peut-être pas implémentée, car elle correspond à une fonctionnalité optionnelle de l'application. Les clés étrangères permettent de représenter les relations entre les différentes tables. Ainsi, un trajet est associé à une ligne et à l'utilisateur qui l'a créé, tandis qu'une ligne est associée à une gare de départ et une gare de terminus. Si elle est implémentée, la table Réservation permettra de relier un utilisateur à un trajet et d'enregistrer le moment de sa réservation.